EEBus Initiative e.V.
Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

EEBus Test Specification

SPINE

Version 1.1.0

Cologne, 2026-09-23

Terms of use for publications of EEBus Initiative e.V.

General information
The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability
EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation
The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Table of contents

Table of contents 3
List of tables 4
1 Introduction 6
1.1 References 6
1.1.1 EEBus documents 6
1.1.2 Normative references 6
1.2 Terms, definitions and abbreviations 7
2 Test suite convention 8
2.1 Test suite identifiers 8
2.2 Preconditions 8
2.3 Test case description 11
2.4 Phrases used throughout the test case definitions 13
2.5 Parameters for the test execution 14
2.5.1 Parameter structure 14
2.5.2 Variable prefixes 15
2.5.3 Address variable naming convention 15
2.5.4 Variable registry 15
2.5.5 Implicit address routing 17
2.5.6 Default parameter block and inheritance 17
2.5.7 PAR_ blocks 18
2.5.8 Manufacturer declarations and Parameter Sheet (PARAM_SHEET) 24
2.6 Acceptance criteria 25
3 Requirement overview and mapping to test cases 27
3.1 Requirement to test case mapping 27
3.2 Test case to requirement mapping 30
4 Test cases 32
4.1 Compatibility 32
4.1.1 TC_SPINE_COMP_001: Reject unknown Function 32
4.1.2 TC_SPINE_COMP_002: Forward version compatibility 32
4.1.3 TC_SPINE_COMP_003: Ignore unknown Elements 33
4.1.4 TC_SPINE_COMP_004: Ignore invalid replies 33
4.1.5 TC_SPINE_COMP_005: Strict version formatting 34
4.1.6 TC_SPINE_COMP_006: Version format variations 35
4.2 Datagram & message counters 36
4.2.1 TC_SPINE_DATA_001: Ascending message counters 36
4.2.2 TC_SPINE_DATA_002: Allow skipped counters 36
4.2.3 TC_SPINE_DATA_003: Handle message counter reset or overflow 37
4.2.4 TC_SPINE_DATA_004: Match reference counter 37
4.2.5 TC_SPINE_DATA_005: Acknowledge notify datagrams 38
4.2.6 TC_SPINE_DATA_006: No response to results 38
4.2.7 TC_SPINE_DATA_007: Ignore ackRequest in read 39
4.2.8 TC_SPINE_DATA_008: No response to results (DUT initiated) 39
4.3 Functional commissioning (Primary NodeManagement) 40
4.3.1 TC_SPINE_FC_001: Reject non-primary destinations 40
4.4 Detailed Discovery 40

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.4.1 TC_SPINE_DDISC_001: Initial discovery of unknown partner 40
4.4.2 TC_SPINE_DDISC_002: Disconnect uncommunicative partner 41
4.5 Binding 42
4.5.1 TC_SPINE_BIND_001: Deny NodeManagement binding 42
4.5.2 TC_SPINE_BIND_002: Reject unbound write 42
4.6 Subscription 43
4.6.1 TC_SPINE_SUBS_001: Accept NodeManagement subscription 43
4.6.2 TC_SPINE_SUBS_002: Idempotent subscription deletion 44
4.7 Entity settings 45
4.7.1 TC_SPINE_ENTITY_001: Dynamic server discovery 45
4.7.2 TC_SPINE_ENTITY_002: Dynamic server subscription 46
4.8 Runtime behavior of server 47
4.8.1 TC_SPINE_RTS_001: Tolerate arbitrary client Feature types 47
4.8.2 TC_SPINE_RTS_002: Tolerate unknown Feature types 48
4.8.3 TC_SPINE_RTS_003: Deduce client address from client's binding request 48
4.8.4 TC_SPINE_RTS_004: Ignore unknown payload Elements 49
4.8.5 TC_SPINE_RTS_005: Apply RFE merge logic ScaledNumberType 50
4.9 Runtime behavior of client 51
4.9.1 TC_SPINE_RTC_001: Ignore extra server Elements 51
4.9.2 TC_SPINE_RTC_002: Ignore unknown server Elements 52
4.9.3 TC_SPINE_RTC_003: Ignore useCaseAvailable flag 52

List of tables

Table 1: Variable registry 17
Table 2: Acceptance criteria 26
Table 3: Requirement mapping 29
Table 4: Test case to requirement mapping 31
Table 5: TC_SPINE_COMP_001 - Test steps 32
Table 6: TC_SPINE_COMP_002 - Test steps 32
Table 7: TC_SPINE_COMP_003 - Test steps 33
Table 8: TC_SPINE_COMP_004 - Test steps 34
Table 9: TC_SPINE_COMP_005 - Test steps 34
Table 10: TC_SPINE_COMP_006 - Test steps 35
Table 11: TC_SPINE_DATA_001 - Test steps 36
Table 12: TC_SPINE_DATA_002 - Test steps 36
Table 13: TC_SPINE_DATA_003 - Test steps 37
Table 14: TC_SPINE_DATA_004 - Test steps 38
Table 15: TC_SPINE_DATA_005 - Test steps 38
Table 16: TC_SPINE_DATA_006 - Test steps 39
Table 17: TC_SPINE_DATA_007 - Test steps 39
Table 18: TC_SPINE_DATA_008 - Test steps 40
Table 19: TC_SPINE_FC_001 - Test steps 40
Table 20: TC_SPINE_DDISC_001 - Test steps 41
Table 21: TC_SPINE_DDISC_002 - Test steps 41

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Table 22: TC_SPINE_BIND_001 - Test steps 42
Table 23: TC_SPINE_BIND_002 - Test steps 43
Table 24: TC_SPINE_SUBS_001 - Test steps 44
Table 25: TC_SPINE_SUBS_002 - Test steps 44
Table 26: TC_SPINE_ENTITY_001 - Test steps 46
Table 27: TC_SPINE_ENTITY_002 - Test steps 47
Table 28: TC_SPINE_RTS_001 - Test steps 47
Table 29: TC_SPINE_RTS_002 - Test steps 48
Table 30: TC_SPINE_RTS_003 - Test steps 49
Table 31: TC_SPINE_RTS_004 - Test steps 50
Table 32: TC_SPINE_RTS_005 - Test steps 51
Table 33: TC_SPINE_RTC_001 - Test steps 51
Table 34: TC_SPINE_RTC_002 - Test steps 52
Table 35: TC_SPINE_RTC_003 - Test steps 53

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

1 Introduction

This document defines the conformance test procedures for the SPINE (Smart Premises Interoperable Neutral-message Exchange) protocol and resource specifications. Unlike use case-specific test specifications, this document focuses purely on the underlying communication and management layer to guarantee cross-manufacturer interoperability.

The primary objective is to verify that a Device Under Test (DUT) correctly implements the foundational SPINE mechanisms. The test cases in this document cover the following core areas:

- Datagram & message integrity: Correct handling of msgCounter, msgCounterReference, and protocol-level acknowledgements.
- Functional commissioning: Proper execution of primary NodeManagement Functions required to establish interoperability. This encompasses discovering remote structural topologies (Entities, Features) via Detailed Discovery, identifying supported functionalities via Use Case discovery, as well as establishing and maintaining logical communication relationships via subscription and binding management.
- Protocol compatibility: Verification of forward/backward compatibility, handling of unknown Elements, and Restricted Function Exchanges (RFE) (partial read/write).

Successful execution of these tests ensures that the DUT provides a reliable and interoperable foundation for any subsequent EEBUS application-layer use cases.

1.1 References

1.1.1 EEBus documents

The following EEBUS specifications and technical documents form the basis for this test specification.

[SPINE_PS]	*Minimum*: EEBus_SPINE_V1.3.0_Final.zip -> EEBus_SPINE_TS_ProtocolSpecification.pdf
[SPINE_RS]	*Minimum*: EEBus_SPINE_V1.3.0_Final.zip -> EEBus_SPINE_TS_ResourceSpecification.pdf
[SHIP]	*Minimum*: EEBus_SHIP_TS_Specification_v1.0.1.pdf *Recommended*: EEBus_SHIP_TS_Specification_v1.1.0.pdf
[UC_LPC]	EEBus_UC_TS_LimitationOfPowerConsumption_V1.0.0.pdf
[UC_LPP]	EEBus_UC_TS_LimitationOfPowerProduction_V1.0.0.pdf
[IG-SPINE]	EEBus_SPINE_IG_ProtocolAndResourceGuidelines_V1.0.0.pdf

1.1.2 Normative references

The normative references listed below contain provisions which, through reference in this text, constitute provisions of this specification.

[RFC 2119]	IETF RFC 2119: Key words for use in RFCs to indicate requirement levels, 1997

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

1.2 Terms, definitions and abbreviations

In this document, the following terms and abbreviations apply:

Binding
A functional connection between matching features (Client to Server) that establishes operational permissions.

CF
Abbreviation for "Configuration". Identifier for a specific precondition state required before test execution.

CS
Abbreviation for "Controllable System". A common generic actor role used in testing.

Detailed Discovery
The process by which a SPINE node learns the complete hierarchical structure (Entities, Features, Supported Functions) of a communication partner.

DUT
Abbreviation for "Device Under Test". The specific implementation being verified for SPINE compliance.

EG
Abbreviation for "Energy Guard". A common generic actor role used in testing.

NodeManagement
A special, mandatory feature (located at Entity 0, Feature 0) responsible for device discovery and functional commissioning.

RFE
Abbreviation for "Restricted Function Exchange". A mechanism in SPINE that allows devices to perform partial data updates instead of exchanging the full Function.

SHIP
Abbreviation for "Smart Home IP".

SPINE
Abbreviation for "Smart Premises Interoperable Neutral-message Exchange".

Subscription
A mechanism allowing a client feature to receive unsolicited update messages (Notifications) from a Server feature.

TC
Abbreviation for "Test Case".

Use Case discovery
The process allowing a device to discover which use cases are supported and which actor a remote device embodies.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2 Test suite convention

This chapter defines the technical conventions and formal rules required for the consistent implementation of the SPINE test suite. It establishes the naming schemes, system configurations, and data variables necessary for both manual and automated test execution.

2.1 Test suite identifiers

To ensure consistency and machine-readability, this test suite uses strict, EEBUS-specific hierarchical naming conventions for all components. The identifiers are self-descriptive:

- Test Case (TC): Named as TC_SPINE_<context>_<number> (Example: TC_SPINE_BIND_001)
- Precondition (PRE): Preconditions are divided into two distinct naming schemes depending on their scope:
  - System/protocol states:
    - Named as PRE_SPINE_<context>_<state> to define a specific protocol state that SHALL be reached, involving the DUT (Example: PRE_SPINE_ConnectionEstablished).
  - Test tool configurations:
    - Named as PRE_TestTool_<configuration> to define a dedicated internal setup of the test tool that does not directly involve the DUT (Example: PRE_TestTool_Multi_EG_Configured).

2.2 Preconditions

This section describes the required initial conditions for test cases. A precondition (PRE) defines a specific system state that SHALL be reached, a functional milestone, or a dedicated configuration of the test tool, which SHALL be fulfilled before the actual test steps can be executed. Preconditions SHALL be executed in the order in which they are listed in the test case.

Note: While system states and functional milestones describe conditions that involve the interaction with the DUT, any dedicated configuration within a precondition refers exclusively to the internal setup of the test tool and not to the DUT. Beyond the state explicitly required by a precondition, the test tool is assumed to behave as a correct SPINE communication partner. Unless a test case explicitly defines otherwise, the test tool SHALL correctly react to any valid, state-appropriate message received from the DUT (e.g., answer a nodeManagementDetailedDiscoveryData read) even where this is not explicitly listed in the individual test steps.

PRE_SPINE_ConnectionEstablished

A stable SPINE communication channel is established and ready for message exchange. This can be at any time after the initial connection and discovery phase.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PRE_SPINE_NewConnectionEstablished

A SPINE connection has been established. At this point, neither the DUT nor the test tool have queried the respective other device for its primary NodeManagement feature.

PRE_SPINE_DetailedDiscovery

The test tool has successfully executed a Detailed Discovery read request and cached the complete logical structure (entities, features, roles) of the DUT.

PRE_SPINE_Scenario_LpcLpp_S1_S3

Successful execution of the initial communication sequence for the currently active limitation scenario (either LPC Scenarios 1 and 3, OR LPP Scenarios 1 and 3). At the protocol level, this requires that the following state is reached between the Controllable System (CS) and the Energy Guard (EG), independent of the specific message order:

- Detailed Discovery and Use Case discovery have been successfully completed.
- Bindings to the LoadControl and DeviceConfiguration server features of the CS are established by the EG.
- Mutual subscriptions to the DeviceDiagnosis server features are established.
- Subscriptions to the LoadControl and DeviceConfiguration server features of the CS are established by the EG.
- The EG has sent at least one deviceDiagnosisHeartbeatData notify and one loadControlLimitListData write request in time, both of which have been acknowledged by the CS with errorNumber = 0.

PRE_SPINE_AddressChangeRecoveryResolved

Following PRE_SPINE_Scenario_LpcLpp_S1_S3, the DUT has stored its established bindings and subscriptions according to its declared PAR_addressChangeRecovery method:

- session-only: the DUT keeps them only for the duration of the uninterrupted communication session (not persistently), or
- persistent-auto-recovery: the DUT keeps them persistently and, after a remote address change, autonomously detects and cleans up the now-invalid entries and re-establishes them towards the changed address.

The declared method is known to the test tool and determines the expected recovery path in the referencing test case.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PRE_TestTool_AsFutureDevice

The test tool simulates a future device by configuring its SPINE stack to announce a protocol version higher than 1.3.0. This higher version MUST strictly remain within the same major version group (e.g., v1.999.999). A higher major version (e.g., v2.0.0) SHALL NOT be used, as the SPINE specification explicitly allows new major releases to break downward compatibility.

PRE_SPINE_SubscriptionEstablished

A SPINE subscription has been successfully established between the test tool and the DUT via nodeManagementSubscriptionRequestCall. The specific direction of the subscription (which device acts as the client requesting the subscription) depends on the roles and requirements of the active test case.

PRE_TestTool_Multi_EG_Configured

The test tool is preconfigured to simulate multiple Energy Guard actors (e.g., two Energy Guards) residing on the same SPINE device. To correctly simulate this topology, the test tool MUST provide at least two distinct logical entities (representing "Client A" and "Client B") that share the exact same SPINE device address but are configured with different entity addresses.

PRE_TestTool_ReplyLpcLppExtraElement_Configured

The test tool is internally pre-configured to autonomously append an additional, standard SPINE Element (e.g., "label") using PAR_replyLpcLppExtraElement to any outgoing loadControlLimitDescriptionListData reply.

PRE_TestTool_ReplyLpcLppUnknownElement_Configured

The test tool is internally pre-configured to autonomously append an entirely unknown Element using PAR_replyLpcLppUnknownElement to any outgoing loadControlLimitDescriptionListData reply.

PRE_TestTool_ReplyUcdAvailableFalse_Configured

The test tool is internally pre-configured to autonomously set the useCaseAvailable flag to false using PAR_replyUcdAvailableFalse in any outgoing nodeManagementUseCaseData reply.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PRE_TestTool_ReplyDiscoveryKnownArbitraryType_Configured

The test tool is internally pre-configured to autonomously reply to any incoming nodeManagementDetailedDiscoveryData read request with its topology using PAR_replyDiscoveryKnownArbitraryType.

PRE_TestTool_ReplyDiscoveryUnknownFutureType_Configured

The test tool is internally pre-configured to autonomously reply to any incoming nodeManagementDetailedDiscoveryData read request with its topology using PAR_replyDiscoveryUnknownFutureType.

2.3 Test case description

To ensure a uniform evaluation of the SPINE protocol requirements, all test cases are presented in a standardized tabular format. This structure ensures that test engineers and developers can clearly distinguish between the protocol state, the technical stimulus, and the required system behavior.

The header of each test case provides the administrative and logical context required for execution:

Test case ID
The unique identifier of the test case according to the strict naming convention (e.g., TC_SPINE_BIND_001).

Test description
A concise summary of the test objective and the specific SPINE protocol mechanism or robustness rule being verified.

Roles
Defines the functional SPINE protocol roles assumed by the DUT and the test tool during the test case execution. Since this is a protocol-level test specification, the roles MUST strictly correspond to the SPINE architecture. Valid values are Server, Client, or Special.
Every certifiable DUT must support both the Server and the Client role (for example, both Actors in LPC/LPP require the Client AND Server role for the heartbeat). The role stated here therefore denotes only the role the DUT assumes within this specific test case; it is not a device-capability filter. Where a test case targets a specific Use Case actor, that actor (Energy Guard (EG) or Controllable System (CS)) SHALL be stated in brackets after the role (e.g., "DUT: Server (CS). Test tool: Client (EG)."). The actor, not the role, determines to which DUTs the test case applies; it is listed in the DUT Actor column of the requirement mapping (Table 4). Test cases with role Special do not target a specific actor and apply to any DUT.
If the test case role is defined as Special (for the test tool or the DUT, or both), the test tool SHALL implicitly set the corresponding addressSource and addressDestination fields in the SPINE header of its outgoing message to {device: <own or of communication partner's device address string>, entity: 0, feature: 0}. Explicit declaration of these addresses in the "PAR_" blocks is therefore not required for NodeManagement communication.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Applicability
Defines the protocol-level prerequisites a DUT MUST fulfill to be eligible for the test. Instead of referencing specific Use Case Actors, the applicability SHALL reference the required SPINE Features, roles, or structural capabilities needed to execute the test. Example 1: "Applicable to any DUT." Example 2: "Applicable to any DUT acting as Controllable System (CS) for the LoadControl feature (e.g., as required by LPC or LPP scenarios)." Example 3: "Applicable to any DUT acting as Energy Guard (EG) for the LoadControl feature.

Test execution
This field specifies how a test case handles variability, either through dynamically selecting a valid context or through multiple distinct execution runs (iterations).

- Context selection: If multiple valid Use Case contexts are available (e.g., LPC and LPP) and no explicit iteration is demanded, the test tool SHALL dynamically select exactly one supported Use Case to execute the test. The preconditions and parameters MUST be resolved dynamically based on the chosen context. A single successful execution in this context is sufficient to reach a positive overall test verdict.
- Explicit iterations: If the test case explicitly demands multiple execution runs (e.g., to test a list of different parameters or error codes), all defined test runs MUST be completed successfully to reach a positive overall test verdict for the test case. Preconditions apply to each individual test run unless stated otherwise.

Referenced requirements
References the centralized requirement identifiers (e.g., [SPINE-TS-BIND-01]) defined in Chapter 3.1, establishing full traceability to the underlying EEBUS specifications.

Precondition
A numbered list of the required initial system states (e.g., PRE_SPINE_ConnectionEstablished) and/or logical configurations that SHALL be successfully established before the first test step can be executed. Each entry references a precondition defined in section 2.2, and the numbering reflects the order in which the preconditions are established.

Test Step [X]
Sequentially numbered execution steps. Each step is strictly divided into two distinct parts:

Action/Execution
The technical stimulus or operation triggered by the test tool (e.g., sending a specific SPINE message).

Expected Result
The exact, observable protocol-level reaction required from the DUT (e.g., acknowledging a request with a resultData message indicating errorNumber = 0, or actively sending a specific discovery request).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2.4 Phrases used throughout the test case definitions

Since this specification defines behavior on a strict protocol level, the test steps avoid tool-specific automation keywords. Instead, a standardized execution notation is used to describe the interactions between the test tool and the DUT.

Execution verbs:

"**sends**"
The test tool actively constructs and transmits a SPINE datagram with the specified <FUNCTION> and cmdClassifier.

"**waits for**"
The test tool passively listens on the active SHIP connection for an incoming SPINE datagram matching the specified criteria.

"**rejects**"
The DUT correctly identifies a protocol or validation error and answers with a resultData message indicating an application error (errorNumber > 0).

"**ignores**"
The DUT processes the received datagram without triggering an error, does not drop the connection, and proceeds with the normal control flow.

"**responds with**"
The DUT sends a SPINE datagram as a reaction to a previously received message. To be considered as response to the received message, the datagram's msgCounterReference SHALL exactly match the msgCounter of the received message.

"**remains silent**"
The DUT does not send any further SPINE datagram in response to the action of the current test step during the applicable timeout period. This means that the test tool verifies that the DUT sends no further datagram where the msgCounterReference matches the msgCounter of the triggering message. Other messages of the DUT (i.e. messages without msgCounterReference or with a different value than the msgCounter of the triggering message) are still permitted and not considered for the criterion to "remain silent". The underlying SHIP connection SHALL remain stable and active.

"**does not respond to**"
The DUT does not send a SPINE datagram (or a specifically restricted type of datagram, e.g., a resultData message) as a reaction to the action of the current test step during the applicable timeout period. A received datagram is considered a reaction to the triggering message if its msgCounterReference exactly matches the msgCounter of that received message. The underlying SHIP connection SHALL remain stable and active.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Message classifiers

When a test step specifies an action like "sends a read request", it strictly refers to the cmdClassifier Element in the SPINE header as defined in [SPINE_PS] Chapter 5.2.4 (i.e., read, reply, notify, write, call, result).

Timeout rules:

- Tolerance window: For all timeouts defined in this specification, the test tool SHALL grant a fixed tolerance window of exactly 2 seconds to account for transmission path latencies and processing delays. If the predefined timer and the added tolerance window expire without the expected datagram arriving, the test tool SHALL mark the test case as failed, unless the specific test step or the acceptance criteria explicitly define a different outcome (e.g., generating a warning instead).
- Default timeout: Unless explicitly specified otherwise in the test step's "Expected Result" column, a standard timeout of 10 seconds applies to all expected responses or incoming messages from the DUT.
- Explicit timeouts: If a test case defines a specific timeout (e.g., "within 30s" or "within 60s"), this explicit value overrides the default timeout.

2.5 Parameters for the test execution

This chapter defines everything the test tool needs to build and address the SPINE datagrams used in the test cases. It is organised as follows: the parameter structure and variable prefixes, the address variable naming convention and the variable registry; the implicit address routing and the default parameter block from which all parameter blocks inherit; the individual PAR_ blocks; and the manufacturer declarations provided via the Parameter Sheet.

2.5.1 Parameter structure

This section specifies the configuration sets for the test keywords (prefixed by "PAR_"). To ensure a clear distinction between protocol-specific data and test automation instructions, the parameters are divided into the structural components of a SPINE datagram and explicit validation assertions.

- HEADER: Represents Elements within datagram.header (e.g., cmdClassifier, msgCounter, ackRequest).
- RFE: Represents Elements within datagram.payload.cmd.filter used for Restricted Function Exchange (e.g., cmdControl, <SELECTORS>, <ELEMENTS>).
- CMD: Represents the actual Function data within datagram.payload.cmd.<FUNCTION>. The required Function name SHALL be stated first, followed by its Elements.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2.5.2 Variable prefixes

To ensure test scripts can be executed flexibly across different DUTs from various manufacturers, dynamic parameters and device-specific values are abstracted using variables. The test automation framework MUST resolve these variables at runtime based on their specific prefix:

- ${CACHE.<variable>}: Represents dynamic data that the test execution engine MUST acquire and store (cache) during the runtime of the test case. This data is typically gathered during the precondition or discovery phases (e.g., Detailed Discovery, Use Case discovery).
    - Example: ${CACHE.Discovered_LpcLpp_LimitId} requires the script to extract the exact limitId from a previously received loadControlLimitDescriptionListData reply and reuse it in the current message.
- ${PARAM_SHEET.<variable>}: Represents static, manufacturer-specific configuration data. Before executing the test suite, the test engineer MUST provide these values via "Parameter Sheet".
    - Example: ${PARAM_SHEET.PAR_lpcLpp_Test_Value1} instructs the test engine to load the concrete test limit value from the provided manufacturer configuration.
- ${SYS.<variable>}: Represents runtime system variables generated directly by the test execution engine (the test tool) at the exact moment the datagram is built.
    - Example: ${SYS.CurrentTime} requires the test script to insert the current system timestamp formatted as a valid SPINE AbsoluteOrRelativeTimeType (e.g., "2026-04-15T13:10:00.0Z") when sending a deviceDiagnosisHeartbeatData notification.
- ${SYS.Unknown_Element_Tag}: The test tool SHALL dynamically generate a random XML Element name for this variable at runtime. To guarantee that this Element does not collide with any current or future SPINE standard Element, the generated string SHALL consist of exactly 15 random characters containing only upper and lower case letters (a-z, A-Z).

2.5.3 Address variable naming convention

To make the exact content of each variable unambiguous at every point of use, the following suffix convention applies:

- _Address: contains a complete SPINE address consisting of all three parts {device, entity, feature}.
- _DeviceAddress: contains only the device part of a SPINE address (a single string).
- _Entity: contains only the entity-part of the address.

All runtime, cache and parameter-sheet variables used in this specification are defined in the Variable registry (Table 1). Test cases SHALL reference only variables listed there and MUST NOT introduce undefined variables.

2.5.4 Variable registry

This registry lists every variable used in the parameter blocks and test steps of this specification. For each variable it states its structure (a full SPINE address {device, entity, feature}, only the device part, an entity number, or a scalar value) together with its content and runtime source.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Variable	Structure	Content
SYS.TestTool_Server_Address	full address	Test tool's own address when acting in the Server role (used as addressSource).
SYS.TestTool_ClientA_Address	full address	Address of test-tool client instance A (multi-client tests).
SYS.TestTool_Client_Address_LpcLpp	full address	Test-tool client address used for LPC/LPP binding/subscription operations.
SYS.TestTool_DeviceAddress	device part	device string of the test tool; inserted inside {device: …}.
SYS.TestTool_Previous_Entity	entity part	Baseline entity address of the test tool before an address shift.
SYS.TestTool_Shifted_Entity	entity part	Shifted entity address, defined as TestTool_Previous_Entity + offset (address-change test).
SYS.TestTool_measurementId	integer	measurementId assigned by the test tool.
SYS.Test_Version_String	string	specificationVersion string injected to test version handling (may be intentionally invalid).
SYS.LpcLpp_Direction	enum	"consume" for LPC, "produce" for LPP; derived from the active Use Case (PARAM_SHEET.PAR_limitationUseCase).
SYS.LpcLpp_UseCaseName	string	Active LPC/LPP use case name: "limitationOfPowerConsumption" (LPC) / "limitationOfPowerProduction" (LPP); derived from the active Use Case (PARAM_SHEET.PAR_limitationUseCase).
SYS.CurrentTime	timestamp	Current UTC time as a valid AbsoluteOrRelativeTimeType (e.g., 2026-04-15T13:10:00.0Z).
SYS.Received_MsgCounter	integer	msgCounter of the most recent datagram received from the DUT (used for msgCounterReference).
SYS.Unknown_Element_Tag	string	Random 15-character element name.
SYS.LpcLpp_Test_Value2_Number	integer	Calculated integer = $\lfloor$${PARAM_SHEET.PAR_lpcLpp_Test_Value2} / $10^{\text{${CACHE.Current_Limit_Scale}}}\rfloor$
CACHE.DUT_DeviceAddress	device part	device string of the DUT, obtained during Detailed Discovery.
CACHE.DUT_Client_Address	full address	Address of the DUT client feature that established the active subscription (destination for notifications).
CACHE.DUT_LoadControl_Address	full address	Address of the DUT's LoadControl server feature.
CACHE.DUT_DeviceConfiguration_Address	full address	Address of the DUT's DeviceConfiguration server feature.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Variable	Structure	Content
CACHE.Discovered_LpcLpp_LimitId	integer	limitId extracted from a received loadControlLimitDescriptionListData reply.
CACHE.Current_Limit_Scale	integer	value.scale of the current limit; SHALL default to 0 if the DUT omitted scale.
PARAM_SHEET.PAR_lpcLpp_Test_Value1	integer	Primary valid LoadControl limit value (from the Parameter Sheet).
PARAM_SHEET.PAR_lpcLpp_Test_Value2	integer	Secondary valid LoadControl limit value.
PARAM_SHEET.PAR_spineVersion	string	SPINE version supported by the DUT (minimum "1.3.0").

Table 1: Variable registry

2.5.5 Implicit address routing

Every SPINE address sent by the test tool always includes its device part. Unless a parameter block explicitly defines otherwise, the test tool SHALL set the device part of both header address fields as:

- addressSource.device = ${SYS.TestTool_DeviceAddress} (the sender = test tool)
- addressDestination.device = ${CACHE.DUT_DeviceAddress} (the receiver = DUT)

The entity and feature parts are set per message type and classifier:

- NodeManagement communication (any nodeManagement… Function - Detailed/UseCase Discovery reads, binding requests/deletes, subscription requests/deletes): entity = 0, feature = 0 on both fields:
  - addressSource = {device: ${SYS.TestTool_DeviceAddress}, entity: 0, feature: 0}
  - addressDestination = {device: ${CACHE.DUT_DeviceAddress}, entity: 0, feature: 0}.
- Own read/write/call requests to a DUT application feature: addressDestination = the discovered full address of the targeted DUT feature (already contains ${CACHE.DUT_DeviceAddress}); addressSource carries the test-tool device as above.
- Responses (reply, result): addressDestination = the exact addressSource of the triggering incoming request (contains the DUT device).
- Notifications (notify): addressDestination = the clientAddress the DUT specified in the subscriptionRequest that established the subscription (contains the DUT device).

Explicit declaration of these fields in the HEADER section of a parameter block is only required when intentionally deviating from this normal communication path (e.g., to simulate a request from an unbound client address).

2.5.6 Default parameter block and inheritance

The block PAR_default defines the default HEADER values that apply to every datagram sent by the test tool. It is a template, not a message, and is never sent on its own. Every other PAR_ block inherits all HEADER fields of PAR_default; a block lists a HEADER field only where it deviates from the default, and any field it does not list keeps the PAR_default value. RFE and CMD content is always stated explicitly per block (there is no default payload).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_default

- HEADER: specificationVersion = ${PARAM_SHEET.PAR_spineVersion}
- HEADER: msgCounter = auto-incremented per datagram by the test tool
- HEADER: addressSource = per Implicit address routing (Section 2.5.5)
- HEADER: addressDestination = per Implicit address routing (Section 2.5.5)

2.5.7 PAR_ blocks

The following parameter blocks define the concrete datagrams the test tool sends during the test steps. Each block is referenced by name from the test cases and inherits all HEADER defaults from PAR_default (Section 2.5.6); it therefore lists only the fields that deviate from those defaults, plus its RFE and CMD content.

PAR_readUnknownFunction

- HEADER: addressDestination = {entity: 0, feature: 0}
- HEADER: specificationVersion = "1.999.999"
- HEADER: cmdClassifier = "read"
- CMD: <FUNCTION> = <generated random string [a-z, A-Z] with 20 characters>

PAR_replyDiscoveryUnknownElements

- HEADER: cmdClassifier = "reply"
- HEADER: specificationVersion = "1.999.999"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: entityInformation.description.entityAddress.entity = 99
- CMD: entityInformation.description.entityType = "unsupportedEntityType"
- CMD: featureInformation.description.featureAddress.entity = 99
- CMD: featureInformation.description.featureAddress.feature = 99
- CMD: featureInformation.description.featureType = "unsupportedFeatureType"
- CMD: featureInformation.description.${SYS.Unknown_Element_Tag} (Element strictly NOT defined in [SPINE_RS]) = "testData"

PAR_notifyAck

- HEADER: addressSource = ${SYS.TestTool_Server_Address}
- HEADER: addressDestination = ${CACHE.DUT_Client_Address}
- HEADER: cmdClassifier = "notify"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = deviceDiagnosisHeartbeatData
    - Child Elements set according to [UC_LPC]/[UC_LPP] Function "deviceDiagnosisHeartbeatData" at Actor Energy Guard.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_bindingNm

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementBindingRequestCall
- CMD: bindingRequest.clientAddress = {device: ${SYS.TestTool_DeviceAddress}, entity: 0, feature: 0}
- CMD: bindingRequest.serverAddress = {device: ${CACHE.DUT_DeviceAddress}, entity: 0, feature: 0}

PAR_bindingLpcLppDelete

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementBindingDeleteCall
- CMD: bindingDelete.clientAddress = ${SYS.TestTool_Client_Address_LpcLpp}
- CMD: bindingDelete.serverAddress = ${CACHE.DUT_LoadControl_Address}

PAR_bindingClientAToLoadControl

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementBindingRequestCall
- CMD: bindingRequest.clientAddress = ${SYS.TestTool_ClientA_Address}
- CMD: bindingRequest.serverAddress = ${CACHE.DUT_LoadControl_Address}

PAR_bindingClientAToDeviceConfiguration

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementBindingRequestCall
- CMD: bindingRequest.clientAddress = ${SYS.TestTool_ClientA_Address}
- CMD: bindingRequest.serverAddress = ${CACHE.DUT_DeviceConfiguration_Address}

PAR_subscriptionNm

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementSubscriptionRequestCall
- CMD: subscriptionRequest.clientAddress = {device: ${SYS.TestTool_DeviceAddress}, entity: 0, feature: 0}
- CMD: subscriptionRequest.serverAddress = {device: ${CACHE.DUT_DeviceAddress}, entity: 0, feature: 0}

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_deleteSubscriptionNm

- HEADER: cmdClassifier = "call"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementSubscriptionDeleteCall
- CMD: subscriptionDelete.clientAddress = {device: ${SYS.TestTool_DeviceAddress}, entity: 0, feature: 0}
- CMD: subscriptionDelete.serverAddress = {device: ${CACHE.DUT_DeviceAddress}, entity: 0, feature: 0}

PAR_writeLimitVal

- HEADER: addressSource = ${SYS.TestTool_Client_Address_LpcLpp}
- HEADER: addressDestination = ${CACHE.DUT_LoadControl_Address}
- HEADER: cmdClassifier = "write"
- HEADER: ackRequest = true
- RFE: cmdControl = "partial"
- CMD: <FUNCTION> = loadControlLimitListData
- CMD: loadControlLimitData.limitId = ${CACHE.Discovered_LpcLpp_LimitId}
- CMD: loadControlLimitData.isLimitActive = true
- CMD: loadControlLimitData.value.number = ${PARAM_SHEET.PAR_lpcLpp_Test_Value1}
- CMD: loadControlLimitData.value.scale = 0

PAR_replyLpcLppExtraElement

- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = loadControlLimitDescriptionListData
- CMD: loadControlLimitDescriptionData.limitId = ${CACHE.Discovered_LpcLpp_LimitId}
- CMD: loadControlLimitDescriptionData.limitType = "signDependentAbsValueLimit"
- CMD: loadControlLimitDescriptionData.limitDirection = ${SYS.LpcLpp_Direction}
- CMD: loadControlLimitDescriptionData.label = "extra" (Well-defined Element of [SPINE_RS] which is unused in [UC_LPC]/[UC_LPP]).

PAR_replyLpcLppUnknownElement

- HEADER: specificationVersion = "1.999.999"
- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = loadControlLimitDescriptionListData
- CMD: loadControlLimitDescriptionData.limitId = ${CACHE.Discovered_LpcLpp_LimitId}
- CMD: loadControlLimitDescriptionData.limitType = "signDependentAbsValueLimit"
- CMD: loadControlLimitDescriptionData.limitCategory = "obligation"
- CMD: loadControlLimitDescriptionData.limitDirection = ${SYS.LpcLpp_Direction}
- CMD: loadControlLimitDescriptionData.measurementId = ${SYS.TestTool_measurementId}
- CMD: loadControlLimitDescriptionData.unit = "W"

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- CMD: loadControlLimitDescriptionData.scopeType = "activePowerLimit"
- CMD: loadControlLimitDescriptionData.${SYS.Unknown_Element_Tag} (Element strictly NOT defined in [SPINE_RS]) = "testData"

PAR_fcDestE1SourceE0

- HEADER: addressDestination = {entity: 1, feature: 0} (Wrong destination)
- HEADER: cmdClassifier = "call"
- CMD: <FUNCTION> = nodeManagementSubscriptionRequestCall
- CMD: subscriptionRequest.clientAddress = {device: ${SYS.TestTool_DeviceAddress}, entity: 0, feature: 0}
- CMD: subscriptionRequest.serverAddress = {device: ${CACHE.DUT_DeviceAddress}, entity: 0, feature: 0}

PAR_replyInvalidWithAck

- HEADER: cmdClassifier = "reply"
- HEADER: ackRequest = true
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: entityInformation.description.entityAddress.entity = "InvalidStringValue" (Intentionally injecting a string into an xs:unsignedInt field to provoke a schema validation error)

PAR_resultAck

- HEADER: cmdClassifier = "result"
- HEADER: ackRequest = true
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = resultData
- CMD: resultData.errorNumber = 0

PAR_nmDiscoveryFuture

- HEADER: specificationVersion = "1.999.999"
- HEADER: cmdClassifier = "read"
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData

PAR_readAckRequestFalse

- HEADER: cmdClassifier = "read"
- HEADER: ackRequest = false
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_readAckRequestTrue
- HEADER: cmdClassifier = "read"
- HEADER: ackRequest = true
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData

PAR_readInvalidVersionFormat
- HEADER: cmdClassifier = "read"
- HEADER: specificationVersion = ${SYS.Test_Version_String}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData

PAR_shiftedDiagnosisAddress
- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: entityInformation.description.entityAddress.entity = ${SYS.TestTool_Shifted_Entity}
- CMD: featureInformation.description.featureAddress.entity = ${SYS.TestTool_Shifted_Entity}
- CMD: featureInformation.description.featureAddress.feature = 10
- CMD: featureInformation.description.featureType = "DeviceDiagnosis"
- CMD: featureInformation.description.role = "server"

with
- ${SYS.TestTool_Shifted_Entity} = ${SYS.TestTool_Previous_Entity} + 37

PAR_shiftedCsEntityAddress
- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: entityInformation.description.entityAddress.entity = ${SYS.TestTool_Shifted_Entity}
- CMD: featureInformation.description.featureAddress.entity = ${SYS.TestTool_Shifted_Entity}
- CMD: featureInformation.description.featureAddress.feature = 10
- CMD: featureInformation.description.featureType = "LoadControl"
- CMD: featureInformation.description.role = "server"

with
- ${SYS.TestTool_Shifted_Entity} = ${SYS.TestTool_Previous_Entity} + 51

PAR_replyDiscoveryKnownArbitraryType
- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: featureInformation.description.featureType = "DeviceClassification"
- CMD: featureInformation.description.role = "client"

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_replyDiscoveryUnknownFutureType

- HEADER: cmdClassifier = "reply"
- HEADER: specificationVersion = "1.999.999"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: featureInformation.description.featureType = "CompletelyUnknownFeatureType"
- CMD: featureInformation.description.role = "client"

PAR_writeLpcLppUnknownElement

- HEADER: cmdClassifier = "write"
- HEADER: specificationVersion = "1.999.999"
- HEADER: ackRequest = true
- RFE: cmdControl = "partial"
- CMD: <FUNCTION> = loadControlLimitListData
- CMD: loadControlLimitData.limitId = ${CACHE.Discovered_LpcLpp_LimitId}
- CMD: loadControlLimitData.isLimitActive = false
- CMD: loadControlLimitData.${SYS.Unknown_Element_Tag} = "testData" (Element strictly NOT defined in [SPINE_RS])

PAR_writeLpcLppMissingScale

- HEADER: cmdClassifier = "write"
- HEADER: ackRequest = true
- RFE: cmdControl = "partial"
- CMD: <FUNCTION> = loadControlLimitListData
- CMD: loadControlLimitData.limitId = ${CACHE.Discovered_LpcLpp_LimitId}
- CMD: loadControlLimitData.isLimitActive = true
- CMD: loadControlLimitData.value.number = ${SYS.LpcLpp_Test_Value2_Number}

with

- ${SYS.LpcLpp_Test_Value2_Number} =
  $\lfloor$ ${PARAM_SHEET.PAR_lpcLpp_Test_Value2} / $10^{\${CACHE.Current_Limit_Scale}}$ $\rfloor$

where "⌊…⌋" is the mathematical "floor" function which rounds the expression (...) to the nearest integer value that is not greater than the expression.

NOTE: The written number is derived as ⌊ value / 10^{scale} ⌋ because the test tool must not dictate a specific number/scale split and does not know the DUT's scale in advance. This keeps the resulting physical limit close to ${PARAM_SHEET.PAR_lpcLpp_Test_Value2} for any scale. Since number is an integer, the stored value is the largest value representable at the DUT's current scale and may be slightly below the target — this is the inherent granularity of the scale and is expected.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_replyUcdAvailableFalse

- HEADER: cmdClassifier = "reply"
- HEADER: msgCounterReference = ${SYS.Received_MsgCounter}
- CMD: <FUNCTION> = nodeManagementUseCaseData
- CMD: useCaseInformation.useCaseSupport.useCaseName = ${SYS.LpcLpp_UseCaseName}
- CMD: useCaseInformation.useCaseSupport.useCaseAvailable = false

PAR_notifyDiscoveryModifiedAck

- HEADER: cmdClassifier = "notify"
- HEADER: ackRequest = true
- RFE: cmdControl = "partial"
- CMD: <FUNCTION> = nodeManagementDetailedDiscoveryData
- CMD: entityInformation.description.entityAddress.entity = 0
- CMD: entityInformation.description.lastStateChange = "modified"

2.5.8 Manufacturer declarations and Parameter Sheet (PARAM_SHEET)

Unlike the payload-specific parameter blocks above that define the structural contents of regular SPINE datagrams, the following parameters specify static manufacturer declarations, device capabilities, or behavioral policies. These parameters are evaluated by the test suite to dynamically adapt the execution flow, timeouts, or applicability criteria of specific test cases according to the declared implementation of the DUT.

PAR_initialTimeoutSupported: Manufacturer declaration for timeout

DUT: Special. Select if the DUT supports the optional termination of uncommunicative initial connections after 30 seconds according to [SPINE_PS] Annex D.3. Possible choices are "yes" (recommended) or "no".

PAR_addressChangeRecovery: **Manufacturer declaration for the recovery from a changed remote server address**

DUT: Client. Select the methodology the DUT applies to recover from changed remote server addresses after reconnection. Exactly one of the following possibilities must be chosen:

- session-only: The DUT stores subscriptions and bindings only for the duration of an uninterrupted communication session (but not persistently across a communication session). (recommended)
- persistent-auto-recovery: The DUT stores subscriptions and bindings persistently. After a reconnection, the DUT is capable of autonomously detecting invalid subscriptions and bindings, cleaning up this information, and setting up subscriptions and bindings upon the changed remote server addresses accordingly.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_clientDetailedDiscoverySupported: Manufacturer declaration for active Detailed Discovery initiation

DUT: Client. Select if the DUT actively initiates a nodeManagementDetailedDiscoveryData read request after establishing a new connection. Possible choices are "yes" or "no".

PAR_clientSubscriptionSupported: Manufacturer declaration for active subscription initiation

DUT: Client. Select if the DUT actively establishes subscriptions to remote server features. Possible choices are "yes" or "no".

PARAM_SHEET Variables: Required Parameter Sheet variables for testing

To execute the test cases successfully, the test engineer MUST provide the following static values regarding the DUT's capabilities:

- ${PARAM_SHEET.PAR_spineVersion}: The explicitly supported SPINE version of the DUT ("1.3.0" or higher).
- ${PARAM_SHEET.PAR_limitationUseCase}: The chosen Use Case for some test cases defined within this test specification (must be "LPC" or "LPP").
- ${PARAM_SHEET.PAR_lpcLpp_Test_Value1}: The first valid limit value within the physical range of the DUT. The value must be an integer.
- ${PARAM_SHEET.PAR_lpcLpp_Test_Value2}: A second, different valid and acceptable limit value for the DUT. The value must be an integer.

2.6 Acceptance criteria

To ensure a high level of safety, reliability, and automated executability, each test case is evaluated based on global definitive criteria. A test is only considered PASSED if all applicable criteria in this section and all step-specific expected results are met simultaneously.

Implicit validity of DUT messages

As a fundamental prerequisite, every SPINE datagram sent by the DUT MUST be completely valid according to the SPINE Protocol and Resource Specifications (including strict XSD schema validation), unless a test case explicitly demands otherwise. To prevent redundancy and logical ambiguity, the word "valid" is intentionally omitted from the expected results of the individual test cases.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Category	Criteria for PASS	Criteria for FAIL
1. Message verification	The DUT sends the expected SPINE message and all VERIFY conditions from the active parameter group are strictly met.	The DUT sends an unexpected message type, or the payload violates the VERIFY conditions.
2. Timeout compliance	The DUT completely executes the expected action and sends the corresponding message within the specified timeout (including the globally defined tolerance window).	The predefined timeout and the added tolerance window expires without the expected reaction from the DUT.
3. Application errors	The DUT responds with resultData.errorNumber == 0 (application success), unless the "Expected result" explicitly demands a specific error code or explicitly requires the DUT to remain silent.	The DUT sends an unexpected resultData message with errorNumber > 0 (application error), unless the "Expected result" explicitly demands this specific error code. Additionally, a FAIL is triggered if the DUT fails to send a resultData message when it is explicitly requested (e.g., via ackRequest = true) and mandated by the protocol.
4. Connection stability	The underlying SHIP connection remains stable and active during the entire test case execution, unless the test case's "Expected result" explicitly permits or requires a connection termination (e.g., during negative tests).	The DUT unexpectedly closes the SHIP connection (when not provoked by a specific negative test action) or stops responding to protocol-level handshakes.
5. Message sequence	The DUT adheres strictly to the required protocol sequences (e.g., msgCounter rules, proper acknowledgement).	The DUT violates sequence rules (e.g., sending a resultData message in response to another resultData message).
6. RFE merge logic	The DUT adheres to RFE rules (e.g., an omitted Element in a test tool's write command is not blindly removed or replaced by a default value on the DUT).	The DUT violates RFE rules (e.g., an omitted Element in a test tool's write command is blindly removed or replaced by a default value on the DUT).
7. Constraint adherence	The test tool automatically verifies that no xData transmitted by the DUT violates its previously communicated xConstraintsListData boundaries.	The DUT transmits xData containing values that violate its own constraints (e.g., sending a limit of 32A when max is 16A).

Table 2: Acceptance criteria

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3 Requirement overview and mapping to test cases

This chapter provides the bidirectional traceability mapping between the normative requirements of the EEBUS specifications and the test cases defined in this document. This mapping ensures that all protocol requirements are fully covered and assists in impact analysis to determine exactly which specification clauses are violated in the event of a specific test failure.

3.1 Requirement to test case mapping

This section establishes the identification system for the foundational protocol requirements. To ensure seamless traceability, all requirements are mapped to their normative sources.

The "Requirement (excerpt)" column deliberately paraphrases each requirement rather than quoting it verbatim. This is intentional: the normative wording resides solely in the referenced source documents (column "Source Reference"), and reproducing it here would risk divergence whenever those documents are updated. The excerpt serves only to identify and summarise the requirement; in case of doubt, the referenced source is authoritative.

Req-ID	Source Reference	Requirement (excerpt)	Test case ID
[SPINE-TS-BIND-01]	[SPINE_PS] Chapter 7.3.1 & 7.3.2	DUT denies binding requests targeting its primary NodeManagement feature (Role "special" is not permitted for bindings).	TC_SPINE_BIND_001
[SPINE-TS-BIND-02]	[SPINE_PS] Chapter 7.3.5 [UC_LPC] Chapter 3.3.3 [UC_LPP] Chapter 3.3.3	DUT rejects write requests with errorNumber 9 if the target feature requires a binding (e.g., LoadControl) but no binding exists.	TC_SPINE_BIND_002
[SPINE-TS-SUBS-01]	[SPINE_PS] Chapter 7.4.1 & 7.5.1	DUT SHALL accept subscription requests to its primary NodeManagement feature (Role "special" exception).	TC_SPINE_SUBS_001
[SPINE-TS-SUBS-02]	[SPINE_PS] Chapter 7.4.4	DUT successfully processes the release (deletion) of an existing subscription.	TC_SPINE_SUBS_002
[SPINE-TS-COMP-01]	[SPINE_RS] Table 19	DUT responds to an unknown SPINE Function with errorNumber 6 (Command not supported).	TC_SPINE_COMP_001
[SPINE-TS-COMP-02]	[SPINE_PS] Chapter 4.3.4.3 & 4.3.4.7	DUT accepts datagrams with a higher specificationVersion (Forward compatibility).	TC_SPINE_COMP_002 TC_SPINE_RTS_002 TC_SPINE_RTS_004 TC_SPINE_RTC_002

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Req-ID	Source Reference	Requirement (excerpt)	Test case ID
[SPINE-TS-COMP-03]	[SPINE_PS] Chapter 4.3.4	DUT silently ignores unknown Elements within a valid datagram.	TC_SPINE_COMP_003 TC_SPINE_RTS_004 TC_SPINE_RTC_001 TC_SPINE_RTC_002
[SPINE-TS-COMP-04]	[SPINE_PS] Chapter 5.2.4 Table 2 & 5.2.5.1	DUT strictly respects the acknowledgement scope of 'reply' messages (Transmission success or error) and does not respond with Application Errors (errorNumber > 0) to invalid payloads.	TC_SPINE_COMP_004
[SPINE-TS-COMP-05]	[IG-SPINE] Chapter 2.5 [SPINE_RS] Chapter 3.10.1.4	Strict adherence to version formatting: All transmitted version strings in the datagram header, Detailed Discovery, and Use Case discovery payloads SHALL exactly match the regular expression "[1-9][0-9]*\.[0-9]+\.[0-9]+". Furthermore, the useCaseDocumentSubRevision field SHALL be set during Use Case discovery.	TC_SPINE_COMP_005
[SPINE-TS-COMP-06]	[IG-SPINE] Chapter 2.5	Applications that receive version strings that do not comply to the formatting rules MAY abort the communication with that other device. As an exception, devices MAY optionally tolerate a leading "v" or "V" on reception.	TC_SPINE_COMP_006
[SPINE-TS-DATA-01]	[SPINE_PS] Chapter 5.2.3.1	Datagrams SHALL have strictly ascending msgCounter values.	TC_SPINE_DATA_001
[SPINE-TS-DATA-02]	[SPINE_PS] Chapter 5.2.3.1	Skipping values in ascending msgCounters is permitted.	TC_SPINE_DATA_002
[SPINE-TS-DATA-03]	[SPINE_PS] Chapter 5.2.3.1	DUT handles unexpectedly low msgCounter by resetting its baseline without error.	TC_SPINE_DATA_003
[SPINE-TS-DATA-04]	[SPINE_PS] Chapter 5.2.3.2	Reply datagrams SHALL contain the appropriate msgCounterReference.	TC_SPINE_DATA_004
[SPINE-TS-DATA-05]	[SPINE_PS] Chapter 5.2.5.1	DUT responds with a resultData message to a notify if ackRequest is true.	TC_SPINE_DATA_005 TC_SPINE_RTS_001
[SPINE-TS-DATA-06]	[SPINE_PS] Chapter 5.2.4	A resultData message SHALL NOT be created as any kind of response to a received resultData message.	TC_SPINE_DATA_006 TC_SPINE_DATA_008

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Req-ID	Source Reference	Requirement (excerpt)	Test case ID
[SPINE-TS-DATA-07]	[SPINE_PS] Chapter 5.2.4	DUT SHALL NOT evaluate the ackRequest flag of a received read operation.	TC_SPINE_DATA_007
[SPINE-TS-FC-01]	[SPINE_PS] Chapter 7.1 [IG-SPINE] Chapter 2.3	Incoming NodeManagement reads/calls NOT sent to Entity 0, Feature 0 are rejected.	TC_SPINE_FC_001
[SPINE-TS-DD-01]	[SPINE_PS] Annex D.3	A DUT with client features actively sends an initial nodeManagementDetailedDiscoveryData read request when connecting to a new, unknown partner.	TC_SPINE_DDISC_001
[SPINE-TS-DD-02]	[SPINE_PS] Annex D.3	DUT MAY close the connection after 30 seconds without receiving a proper command or Detailed Discovery request.	TC_SPINE_DDISC_002
[SPINE-TS-UCD-01]	[IG-SPINE] Chapter 2.2	DUT completely ignores the useCaseAvailable flag if it is sent by a server actor during Use Case discovery.	TC_SPINE_RTC_003
[SPINE-TS-ES-01]	[UC_LPC] Chapter 3.4.1 & 3.4.3	DUT discovers and uses server actors dynamically regardless of addresses/identifiers.	TC_SPINE_ENTITY_001
[SPINE-TS-ES-02]	[UC_LPC] Chapter 3.4.3	DUT subscribes to the Energy Guard’s server feature DeviceDiagnosis based on dynamic topology.	TC_SPINE_ENTITY_002
[SPINE-TS-RT-01]	[SPINE_PS] Chapter 3.1 & 7.1.1.4	DUT tolerates client features of any known SPINE feature type.	TC_SPINE_RTS_001
[SPINE-TS-RT-02]	[UC_LPC] Chapter 3.4.1.1 & 3.4.3.1	DUT correctly deduces the relevant feature client based on the incoming binding request.	TC_SPINE_RTS_003
[SPINE-TS-RT-03]	[SPINE_PS] Chapter 5.3.4.8	In case RFE is used, the "merge logic" (omitted = retain existing state) takes precedence over any default rules. A partial write with number present and scale missing SHALL update the number and retain the old scale.	TC_SPINE_RTS_005

Table 3: Requirement mapping

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3.2 Test case to requirement mapping

This section provides a reverse mapping from the test cases defined in this document to the normative requirements. It allows the identification of the specific rules verified by a given test case depending on the role of the device under test (DUT).

DUT Role vs. DUT Actor

The DUT Role column states the SPINE protocol role (Server, Client, or Special) the DUT assumes within the test case. Because every certifiable DUT must support both roles, the DUT Role is not an applicability filter. The DUT Actor column states the Use Case actor a DUT must implement for the test case to apply: EG (Energy Guard), CS (Controllable System), or Any (applies to all DUTs; used for every Special test case). The manufacturer declares the DUT's Actor(s) in the Parameter Sheet; test cases whose Actor does not match are greyed out automatically.

Status definition (M/O)

This column indicates the implementation requirement for the specified DUT Actor:

- M (Mandatory): The requirement SHALL be implemented. The corresponding test case MUST be passed to achieve compliance.
- O (Optional): The requirement MAY be implemented. If the manufacturer chooses to support this feature, the corresponding test case MUST be passed to prove compliance.

Test Case ID	DUT Role	DUT Actor	M/O	Req-ID
TC_SPINE_COMP_001	Special	Any	M	[SPINE-TS-COMP-01]
TC_SPINE_COMP_002	Special	Any	M	[SPINE-TS-COMP-02]
TC_SPINE_COMP_003	Special	Any	M	[SPINE-TS-COMP-03]
TC_SPINE_COMP_004	Special	Any	M	[SPINE-TS-COMP-04]
TC_SPINE_COMP_005	Special	Any	M	[SPINE-TS-COMP-05]
TC_SPINE_COMP_006	Special	Any	M	[SPINE-TS-COMP-06]
TC_SPINE_DATA_001	Special	Any	M	[SPINE-TS-DATA-01]
TC_SPINE_DATA_002	Special	Any	M	[SPINE-TS-DATA-02]
TC_SPINE_DATA_003	Special	Any	M	[SPINE-TS-DATA-03]
TC_SPINE_DATA_004	Special	Any	M	[SPINE-TS-DATA-04]
TC_SPINE_DATA_005	Special	Any	M	[SPINE-TS-DATA-05]
TC_SPINE_DATA_006	Special	Any	M	[SPINE-TS-DATA-06]
TC_SPINE_DATA_007	Special	Any	M	[SPINE-TS-DATA-07]
TC_SPINE_DATA_008	Special	Any	M	[SPINE-TS-DATA-06]
TC_SPINE_FC_001	Special	Any	M	[SPINE-TS-FC-01]
TC_SPINE_DDISC_001	Special	Any	M	[SPINE-TS-DD-01]
TC_SPINE_DDISC_002	Special	Any	M	[SPINE-TS-DD-02]
TC_SPINE_BIND_001	Special	Any	M	[SPINE-TS-BIND-01]
TC_SPINE_BIND_002	Server	CS	M	[SPINE-TS-BIND-02]
TC_SPINE_SUBS_001	Special	Any	M	[SPINE-TS-SUBS-01]

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test Case ID	DUT Role	DUT Actor	M/O	Req-ID
TC_SPINE_SUBS_002	Special	Any	M	[SPINE-TS-SUBS-02]
TC_SPINE_ENTITY_001	Client	EG	M	[SPINE-TS-ES-01]
TC_SPINE_ENTITY_002	Client	CS	M	[SPINE-TS-ES-02]
TC_SPINE_RTS_001	Server	CS	M	[SPINE-TS-RT-01] [SPINE-TS-DATA-05]
TC_SPINE_RTS_002	Server	CS	M	[SPINE-TS-COMP-02]
TC_SPINE_RTS_003	Server	CS	M	[SPINE-TS-RT-02]
TC_SPINE_RTS_004	Server	CS	M	[SPINE-TS-COMP-02] [SPINE-TS-COMP-03]
TC_SPINE_RTS_005	Server	CS	M	[SPINE-TS-RT-03]
TC_SPINE_RTC_001	Client	EG	M	[SPINE-TS-COMP-03]
TC_SPINE_RTC_002	Client	EG	M	[SPINE-TS-COMP-02] [SPINE-TS-COMP-03]
TC_SPINE_RTC_003	Client	EG	M	[SPINE-TS-UCD-01]

Table 4: Test case to requirement mapping

Note on test execution

The actual execution of a test case depends strictly on the "Applicability" criteria defined within the respective test case description. If a DUT does not meet the functional or structural prerequisites of a test case (e.g., a client-specific test case is not applicable to a pure server DUT), the execution requirement does not apply. In such scenarios, the test case SHALL be skipped and assessed as "Not Applicable", regardless of its "M" or "O" status in this matrix.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4 Test cases

4.1 Compatibility

4.1.1 TC_SPINE_COMP_001: Reject unknown Function

Test case ID: TC_SPINE_COMP_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-01]

Description: This test case verifies that the DUT rejects a read request for an entirely unknown SPINE Function with an error (errorNumber > 0). To guarantee that the request reaches a valid endpoint and is not rejected beforehand due to an unknown address, the test specifically targets the mandatory primary NodeManagement feature. As a best practice, the DUT SHOULD use errorNumber = 6 (Function not supported).

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a read request using PAR_readUnknownFunction.	The DUT responds with a resultData message indicating an errorNumber > 0 (best practice: errorNumber = 6).

Table 5: TC_SPINE_COMP_001 - Test steps

4.1.2 TC_SPINE_COMP_002: Forward version compatibility

Test case ID: TC_SPINE_COMP_002

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-02]

Description: Verifies that the DUT accepts and processes SPINE messages with a higher minor version for known Functions.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData using PAR_nmDiscoveryFuture	The DUT responds with a nodeManagementDetailedDiscoveryData reply.

Table 6: TC_SPINE_COMP_002 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.1.3 TC_SPINE_COMP_003: Ignore unknown Elements

Test case ID: TC_SPINE_COMP_003

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-03]

Description: This test case verifies that the DUT accepts a SPINE datagram containing unknown enumerations and class Elements (as they could be introduced in a future SPINE version) during the Detailed Discovery phase. Specifically, it tests the tolerance for unknown Elements within a nodeManagementDetailedDiscoveryData reply. The DUT silently ignores the unknown content and continues the communication using the remaining known content.

Applicability: Applicable to any DUT if the manufacturer declares PAR_clientDetailedDiscoverySupported = "yes". Not applicable to pure server DUTs.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool waits for nodeManagementDetailedDiscoveryData read request for up to 30s.	The DUT sends a nodeManagementDetailedDiscoveryData read request.
2	The test tool responds with a nodeManagementDetailedDiscoveryData reply using PAR_replyDiscoveryUnknownElements.	The DUT does not respond to the reply with a resultData message.

Table 7: TC_SPINE_COMP_003 - Test steps

4.1.4 TC_SPINE_COMP_004: Ignore invalid replies

Test case ID: TC_SPINE_COMP_004

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-04]

Description: This test case verifies that the DUT does not respond to an invalid payload with an application error (errorNumber > 0).

Applicability: Applicable to any DUT if the manufacturer declares PAR_clientDetailedDiscoverySupported = "yes". Not applicable to pure server DUTs.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool waits for nodeManagementDetailedDiscoveryData read request for up to 30s.	The DUT sends a nodeManagementDetailedDiscoveryData read request.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
2	The test tool responds with a nodeManagementDetailedDiscoveryData reply using PAR_replyInvalidWithAck.	The DUT remains silent or responds with a resultData message (ACK) indicating errorNumber = 0, but no application error (errorNumber > 0).

Table 8: TC_SPINE_COMP_004 - Test steps

4.1.5 TC_SPINE_COMP_005: Strict version formatting

Test case ID: TC_SPINE_COMP_005

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-05]

Description: This test case verifies that the DUT strictly adheres to the version formatting rules when transmitting messages. When requested by the test tool, the DUT MUST provide a nodeManagementDetailedDiscoveryData reply and a nodeManagementUseCaseData reply where all version strings (in the datagram header and the payloads) strictly match the required SpecificationVersionType regular expression. Furthermore, it verifies that the useCaseDocumentSubRevision is populated during Use Case discovery as required by the SPINE Implementation Guideline.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request.	The DUT responds with a nodeManagementDetailedDiscoveryData reply. The test tool verifies that the value of datagram.header.specificationVersion AND every entry in specificationVersionList.specificationVersion strictly matches the regular expression "[1-9][0-9]*\.[0-9]+\.[0-9]+" (without the surrounding double quotes).
2	The test tool sends a nodeManagementUseCaseData read request.	The DUT responds with a nodeManagementUseCaseData reply. The test tool verifies that every useCaseVersion strictly matches the regular expression "[1-9][0-9]*\.[0-9]+\.[0-9]+" (without the surrounding double quotes), AND that the element useCaseDocumentSubRevision is present and populated.

Table 9: TC_SPINE_COMP_005 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.1.6 TC_SPINE_COMP_006: Version format variations

Test case ID: TC_SPINE_COMP_006

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-COMP-06]

Description: This test case verifies the DUT's behavior when receiving datagrams with various incorrectly formatted or unsupported specificationVersion strings in the header. Due to the transitional nature of version string enforcement in the field, specific deviations from the strict protocol rules are currently tolerated and SHALL trigger a WARNING instead of a test failure, as explicitly defined in the expected results.*[1] Any other protocol violation leads to a FAIL.

Applicability: Applicable to any DUT.

Test execution: The test requires multiple iterations. Since a valid rejection mechanism by the DUT is the termination of the connection, the test tool MUST ensure a new connection is established before each iteration if the previous connection was dropped.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	Execute the following block of sub-steps for each version string in the list ["TS1.3.0", "V0.3.0", "v0.3.0","0.3.0", "2.0.0", "V1.3.0", "v1.3.0"]: a) If the SPINE connection was terminated in a previous iteration, the test tool establishes a new SPINE connection. b) The test tool sends a nodeManagementDetailedDiscoveryData read request using PAR_readInvalidVersionFormat with the header version set to the current string.	For each iteration, the DUT SHALL handle the request in one of the following ways: - (Recommended) The DUT strictly rejects the request by either sending a resultData message indicating an application error (errorNumber > 0) or by terminating the connection. - (Tolerated with WARNING) The DUT accepts the message and responds with a nodeManagementDetailedDiscoveryData reply, or fails to respond without dropping the connection. In this case, the test tool SHALL log a WARNING and the step is considered passed for this transitional phase. Any other behavior violates the expected result and leads to a FAIL.

Table 10: TC_SPINE_COMP_006 - Test steps

*[1] Note: A future version of this test specification may tighten these criteria and enforce a hard FAIL instead of a WARNING for specific invalid version strings.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.2 Datagram & message counters

4.2.1 TC_SPINE_DATA_001: Ascending message counters

Test case ID: TC_SPINE_DATA_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-01]

Description: This test case verifies that the DUT assigns msgCounter values in strictly ascending order for its outgoing messages.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request 20 times consecutively.	The DUT responds with 20 nodeManagementDetailedDiscoveryData replies with strictly ascending msgCounter values.*[1]

Table 11: TC_SPINE_DATA_001 - Test steps

*[1] Note: It is assumed that no msgCounter overflow (wrap-around to 0) occurs during this specific test run.

4.2.2 TC_SPINE_DATA_002: Allow skipped counters

Test case ID: TC_SPINE_DATA_002

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-02]

Description: This test case verifies that the DUT accepts incoming messages even if the msgCounter values from the test tool are not consecutive (skipping values), as long as they remain ascending.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request with msgCounter set to 21012.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.
2	The test tool sends a nodeManagementDetailedDiscoveryData read request with msgCounter set to 123321.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.

Table 12: TC_SPINE_DATA_002 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.2.3 TC_SPINE_DATA_003: Handle message counter reset or overflow

Test case ID: TC_SPINE_DATA_003

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-03]

Description: This test case verifies that the DUT processes datagrams even if the msgCounter is lower than the last received value (e.g., after a sender reboot) and updates its baseline accordingly.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request with msgCounter set to 123456.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.
2	The test tool sends a nodeManagementDetailedDiscoveryData read request with msgCounter set to 1000.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.
3	The test tool sends a nodeManagementDetailedDiscoveryData read request with msgCounter set to 2000.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.

Table 13: TC_SPINE_DATA_003 - Test steps

4.2.4 TC_SPINE_DATA_004: Match reference counter

Test case ID: TC_SPINE_DATA_004

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-04]

Description: This test case verifies that the DUT dynamically and correctly sets the msgCounterReference in its response to match the msgCounter of the original request across a sequence of messages. The test tool waits for the corresponding reply before sending the next request.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	Execute the following block of sub-steps 20 times. The iteration counter *i* starts at 0: a) The test tool sends a nodeManagementDetailedDiscover	For each of the 20 iterations: The DUT responds with a nodeManagementDetailedDiscoveryData reply whose msgCounterReference exactly

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
	yData read request, starting with a msgCounter of 88777 + 200*i. b) The test tool waits for the corresponding reply.	matches the msgCounter of the respective request (88777 + 200*i).

Table 14: TC_SPINE_DATA_004 - Test steps

4.2.5 TC_SPINE_DATA_005: Acknowledge notify datagrams

Test case ID: TC_SPINE_DATA_005

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-05]

Description: This test case verifies that the DUT sends a resultData message in response to an incoming notify datagram if the header.ackRequest flag is set to true.

Applicability: Applicable to any DUT if the manufacturer declares PAR_clientSubscriptionSupported = "yes". Not applicable to pure server DUTs.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_SubscriptionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData notify datagram using PAR_notifyDiscoveryModifiedAck.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0.

Table 15: TC_SPINE_DATA_005 - Test steps

4.2.6 TC_SPINE_DATA_006: No response to results

Test case ID: TC_SPINE_DATA_006

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-06]

Description: The test case verifies that the DUT does not respond to a result datagram, even if the header.ackRequest flag is set to true.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request.	The DUT responds with a nodeManagementDetailedDiscoveryData reply.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
2	The test tool sends a resultData message using PAR_resultAck.	The DUT remains silent and does not send any response. *[1]

Table 16: TC_SPINE_DATA_006 - Test steps

*1 Note: If the DUT did not implicitly or explicitly set ackRequest=true in its reply in Step 1, it might silently discard the resultData message in Step 2 simply because it did not expect a result and already cleared its internal message queue for this transaction. The observable outcome (no response) remains correct, though the exact internal reason for discarding the message cannot be strictly isolated here.

4.2.7 TC_SPINE_DATA_007: Ignore ackRequest in read

Test case ID: TC_SPINE_DATA_007

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-07]

Description: This test case verifies that the DUT correctly ignores the ackRequest Element within a read operation. According to the SPINE protocol, the obligation to reply to a read request is independent of the ackRequest flag, which SHALL NOT be evaluated.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementDetailedDiscoveryData read request using PAR_readAckRequestFalse.	The DUT responds with a reply datagram.
2	The test tool sends a nodeManagementDetailedDiscoveryData read request using PAR_readAckRequestTrue.	The DUT responds with a reply datagram AND does NOT send an additional resultData message (ACK) in response to the read request.

Table 17: TC_SPINE_DATA_007 - Test steps

4.2.8 TC_SPINE_DATA_008: No response to results (DUT initiated)

Test case ID: TC_SPINE_DATA_008

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DATA-06]

Description: This test case verifies that the DUT does not respond to a resultData message, even if the ackRequest flag is set to true and the resultData is sent as a direct response to a request initiated by the DUT itself.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Applicability: Applicable to any DUT if the manufacturer declares PAR_clientDetailedDiscoverySupported = "yes". Not applicable to pure server DUTs.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool waits for a nodeManagementDetailedDiscoveryData read request for up to 30s.	The DUT sends a nodeManagementDetailedDiscoveryData read request.
2	The test tool responds with a resultData message using PAR_resultAck.	The DUT remains silent or terminates the connection. The DUT SHALL NOT send a resultData message.

Table 18: TC_SPINE_DATA_008 - Test steps

4.3 Functional commissioning (Primary NodeManagement)

4.3.1 TC_SPINE_FC_001: Reject non-primary destinations

Test case ID: TC_SPINE_FC_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-FC-01]

Description: This test case verifies that the DUT declines NodeManagement datagrams if the addressDestination is not the primary node management instance (Entity 0, Feature 0).

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementSubscriptionRequestCall using PAR_fcDestE1SourceE0.	The DUT rejects the request and sends a resultData message (NACK) indicating errorNumber > 0.

Table 19: TC_SPINE_FC_001 - Test steps

4.4 Detailed Discovery

4.4.1 TC_SPINE_DDISC_001: Initial discovery of unknown partner

Test case ID: TC_SPINE_DDISC_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DD-01]

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test case verifies that the DUT actively sends an initial nodeManagementDetailedDiscoveryData read request when connecting to a completely new, unknown communication partner. To enforce this behavior and bypass any device-side caching, the test tool simulates an unknown partner by utilizing a fresh, previously unused SPINE device address over a newly established underlying communication channel.

Applicability: Applicable only to a DUT if the manufacturer declares PAR_clientDetailedDiscoverySupported = "yes". Not applicable to pure server DUTs.

Preconditions:

1. PRE_SPINE_NewConnectionEstablished *[1]

Step	Action/Execution	Expected result
1	The test tool waits for a nodeManagementDetailedDiscoveryData read request for up to 30s.	The DUT sends a nodeManagementDetailedDiscoveryData read request.

Table 20: TC_SPINE_DDISC_001 - Test steps

*[1] Note: To guarantee that the DUT treats the test tool as a completely unknown communication partner, the test tool MUST actively present a new identity. This requires the test tool to use a newly generated SPINE device address that has never been connected to the DUT before. Furthermore, the test tool MUST ensure that the underlying communication channel is initiated without relying on any cached session data or existing trust relationships from previous tests.

4.4.2 TC_SPINE_DDISC_002: Disconnect uncommunicative partner

Test case ID: TC_SPINE_DDISC_002

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-DD-02]

Description: This test case verifies that the DUT closes the connection to a newly paired, uncommunicative partner after 30 seconds of inactivity by either actively terminating the SPINE connection or initiating a communication retry.

Applicability: Applicable to any DUT if the manufacturer declares PAR_initialTimeoutSupported = "yes".

Preconditions:

1. PRE_SPINE_NewConnectionEstablished *[1]

Step	Action/Execution	Expected result
1	The test tool waits for a SPINE connection termination for up to 30s plus the globally defined tolerance window.*[2]	The DUT either terminates the SPINE connection actively or initiates a communication retry after 30s of inactivity (within the globally defined tolerance window).

Table 21: TC_SPINE_DDISC_002 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

*[1] Note: The test tool MUST be a newly paired, completely unknown communication partner (i.e., no cached NodeManagement data exists on the DUT).

*[2] Note: During this time, the test tool SHALL NOT send any datagrams (e.g., no Detailed Discovery read) and SHALL silently drop all incoming requests.

4.5 Binding

4.5.1 TC_SPINE_BIND_001: Deny NodeManagement binding

Test case ID: TC_SPINE_BIND_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-BIND-01]

Description: This test case verifies that the DUT denies a binding request to its primary NodeManagement feature.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_DetailedDiscovery

Step	Action/Execution	Expected result
1	The test tool sends a binding request call using PAR_bindingNm.	The DUT rejects the request and sends a resultData message (NACK) indicating errorNumber > 0.
2	The test tool sends a nodeManagementBindingData read request.	The DUT responds with a nodeManagementBindingData reply containing no binding entry for its primary NodeManagement feature. *[1]

Table 22: TC_SPINE_BIND_001 - Test steps

*[1] Note: The test tool MUST strictly verify that the reply contains NO bindingEntry where serverAddress.entity is 0 AND serverAddress.feature is 0. This verification SHALL be performed independently of the presence or content of the device Element.

4.5.2 TC_SPINE_BIND_002: Reject unbound write

Test case ID: TC_SPINE_BIND_002

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-BIND-02]

Description: This test case verifies that the DUT rejects a write request with an application error (errorNumber > 0; errorNumber 9 'Binding is necessary' is recommended) if the target Feature requires a binding but the sender does not hold a valid binding. To verify this mechanism, this test utilizes a power limitation context (LPC or LPP). The test tool explicitly sends a heartbeat before the

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

un-bound write request as write requests are rejected in this context if the last heartbeat arrived too long ago.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_Scenario_LpcLpp_S1_S3

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementBindingDeleteCall using PAR_bindingLpcLppDelete.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0.
2	The test tool sends a deviceDiagnosisHeartbeatData notify using PAR_notifyAck.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0.
3	The test tool sends a loadControlLimitListData write request using PAR_writeLimitVal.	The DUT rejects the request and sends a resultData message (NACK) indicating an application errorNumber > 0 (recommended errorNumber = 9).
4	The test tool sends a nodeManagementBindingData read request.	The DUT responds with a nodeManagementBindingData reply containing no binding entry for its LoadControl Feature.

Table 23: TC_SPINE_BIND_002 - Test steps

4.6 Subscription

4.6.1 TC_SPINE_SUBS_001: Accept NodeManagement subscription

Test case ID: TC_SPINE_SUBS_001

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-SUBS-01]

Description: This test case verifies that the DUT accepts a subscription request to its primary NodeManagement feature and provides the corresponding subscription entry upon request.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_DetailedDiscovery

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementSubscriptionRequestCall using PAR_subscriptionNm.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0.
2	The test tool *sends a nodeManagementSubscriptionData read request*.	The DUT responds with a nodeManagementSubscriptionData reply containing a subscription entry for its primary NodeManagement feature. *[1]

Table 24: TC_SPINE_SUBS_001 - Test steps

*[1] Note: The test tool MUST strictly verify that the reply contains a subscriptionEntry where clientAddress.entity is 0 AND clientAddress.feature is 0 AND serverAddress.entity is 0 AND serverAddress.feature is 0. This verification SHALL be performed independently of the presence or content of the device Element in either address.

4.6.2 TC_SPINE_SUBS_002: Idempotent subscription deletion

Test case ID: TC_SPINE_SUBS_002

Role: DUT: Special. Test tool: Special.

Reference: [SPINE-TS-SUBS-02]

Description: This test case verifies that an existing subscription can be successfully deleted and that the DUT handles duplicate deletion requests gracefully (idempotency) by either confirming the deletion or safely rejecting the redundant command.

Applicability: Applicable to any DUT.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_DetailedDiscovery
3. PRE_SPINE_SubscriptionEstablished *[1]

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementSubscriptionDeleteCall using PAR_deleteSubscriptionNm.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0.
2	The test tool sends a nodeManagementSubscriptionDeleteCall using PAR_deleteSubscriptionNm.	The DUT responds with a resultData message (ACK) indicating errorNumber = 0 or an application error (errorNumber > 0).
3	The test tool sends a nodeManagementSubscriptionData read request.	The DUT responds with a nodeManagementSubscriptionData reply containing no entry for its primary NodeManagement feature. *[2]

Table 25: TC_SPINE_SUBS_002 - Test steps

*[1] Note: This precondition MUST establish a subscription explicitly targeting the DUT's primary NodeManagement feature (Entity 0, Feature 0).

*[2] Note: The test tool MUST strictly verify that the reply contains NO subscriptionEntry where clientAddress.entity is 0 AND clientAddress.feature is 0 AND serverAddress.entity is 0 AND

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

serverAddress.feature is 0. This verification SHALL be performed independently of the presence or content of the device Element in either address.

4.7 Entity settings

4.7.1 TC_SPINE_ENTITY_001: Dynamic server discovery

Test case ID: TC_SPINE_ENTITY_001

Role: DUT: Client (EG). Test tool: Server (CS).

Reference: [SPINE-TS-ES-01]

Description: This test case verifies that the DUT consistently discovers Use Case compliant server Actors regardless of their specific SPINE addresses and identifiers after a reconnection. It ensures that the DUT does not rely on cached or hardcoded addresses and dynamically adapts to a changed topology. Because the test tool changes its entity address during the disconnect, any persistently stored binding or subscription on the DUT becomes invalid. The DUT MUST perform an internal cleanup and re-initiate the communication from scratch. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The DUT assumes the role of an Energy Guard (EG) and the test tool simulates a Controllable System (CS).

Applicability: Applicable to DUTs that assume the role of Energy Guard (EG) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_Scenario_LpcLpp_S1_S3

Step	Action/Execution	Expected result
1	The test tool terminates the SPINE connection and its underlying communication channel. During the interruption, the test tool internally deletes all its binding and subscription entries. It then re-configures itself with a new (different) Entity address and changed identifiers (e.g., limitId) for its active limitation Use Case Features. Finally, it re-enables its discoverability and communication again on the underlying communication channel.	The DUT establishes a new SPINE connection within 300s.
2	The test tool waits for a nodeManagementDetailedDiscoveryData read request.	The DUT sends a nodeManagementDetailedDiscoveryData read request within 30s.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
3	The test tool responds with a nodeManagementDetailedDiscoveryData reply containing the new topology using PAR_shiftedCsEntityAddress and awaits/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 strictly targeting the newly discovered Entity addresses and identifiers within 120s.

Table 26: TC_SPINE_ENTITY_001 - Test steps

4.7.2 TC_SPINE_ENTITY_002: Dynamic server subscription

Test case ID: TC_SPINE_ENTITY_002

Role: DUT: Client (CS). Test tool: Server (EG).

Reference: [SPINE-TS-ES-02]

Description: This test case verifies that the DUT dynamically discovers and subscribes to a required server feature (e.g., DeviceDiagnosis) after a reconnection, regardless of the server's previous SPINE address. The test uses the initial communication logic to establish a baseline, then shifts the test tool's Entity address. Because the test tool changes its Entity address during the disconnect, bindings or subscriptions that might still be stored on the DUT from the previous communication session would not work anymore. The DUT MUST update its communication and subscription towards the changed address after reconnection. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The DUT assumes the role of a Controllable System (CS) and the test tool simulates an Energy Guard (EG).

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_Scenario_LpcLpp_S1_S3
3. PRE_SPINE_AddressChangeRecoveryResolved

Step	Action/Execution	Expected result
1	The test tool terminates the SPINE connection and its underlying communication channel. During the interruption, the test tool internally deletes all its binding and subscription entries. It then re-configures itself with a new (different) Entity address for its Actor Energy Guard*[1,2]. Finally, it re-enables its discoverability and communication again on the underlying communication channel.	The DUT establishes a new SPINE connection within 300s.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
2	The test tool initiated/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s. This includes the subscription of the test tool's shifted DeviceDiagnosis feature.

Table 27: TC_SPINE_ENTITY_002 - Test steps

*[1] Note: This also means that the test tool will report the changed Entity address in its nodeManagementDetailedDiscoveryData and nodeManagementUseCaseData replies and notifications.

*[2] Note: The Energy Guard's DeviceDiagnosis server feature is shifted according to PAR_shiftedDiagnosisAddress. It is on the same entity as the Energy Guard's client functionality.

4.8 Runtime behavior of server

4.8.1 TC_SPINE_RTS_001: Tolerate arbitrary client Feature types

Test case ID: TC_SPINE_RTS_001

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-RT-01], [SPINE-TS-DATA-05]

Description: This test case verifies that the DUT accepts a nodeManagementDetailedDiscoveryData reply and correctly processes subsequent runtime communication, even if the client Actor announces its client Feature with an arbitrary but known SPINE Feature type (e.g., DeviceClassification instead of a client Feature with feature type = “Generic”). To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The test tool (EG) announces an arbitrary featureType, but the DUT (CS) MUST silently tolerate it and proceed with the necessary LPC or LPP commissioning (binding, subscription) and runtime message processing.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_ReplyDiscoveryKnownArbitraryType_Configured
2. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool initiates/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s.

Table 28: TC_SPINE_RTS_001 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.8.2 TC_SPINE_RTS_002: Tolerate unknown Feature types

Test case ID: TC_SPINE_RTS_002

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-COMP-02]

Description: This test case verifies that the DUT accepts a nodeManagementDetailedDiscoveryData reply and correctly processes subsequent runtime communication, even if the client Actor announces an entirely unknown client Feature type. To comply with SPINE protocol rules for forward compatibility, the test tool simulates a future release by setting its SPINE specification version to a higher minor/revision value within the same major release than any currently released version. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The test tool (EG) announces an unknown Feature type, but the DUT (CS) MUST silently tolerate it and proceed with the necessary LPC/LPP commissioning (binding, subscription) and runtime message processing.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_ReplyDiscoveryUnknownFutureType_Configured
2. PRE_TestTool_AsFutureDevice
3. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool initiates/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s.

Table 29: TC_SPINE_RTS_002 - Test steps

4.8.3 TC_SPINE_RTS_003: Deduce client address from client's binding request

Test case ID: TC_SPINE_RTS_003

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-RT-02]

Description: This test ensures that if multiple client Actors are present, the DUT subscribes to the required server feature (e.g., DeviceDiagnosis) of the specific client that actively creates a binding to the DUT's Features. It verifies that the DUT correctly deduces the relevant Feature client based on the incoming binding request and ignores other available client Entities. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The test tool simulates a multi Energy Guard (EG) topology.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_Multi_EG_Configured
2. PRE_SPINE_NewConnectionEstablished
3. PRE_SPINE_DetailedDiscovery

Step	Action/Execution	Expected result
1	The test tool sends a nodeManagementBindingRequestCall from Client A using PAR_bindingClientAToLoadControl.	The DUT responds with a resultData message indicating errorNumber = 0 (ACK).
2	The test tool sends a nodeManagementBindingRequestCall from Client A using PAR_bindingClientAToDeviceConfiguration.	The DUT responds with a resultData message indicating errorNumber = 0 (ACK).
3	The test tool waits for the node management initiation (subscription).	The DUT autonomously performs a Detailed Discovery (handled by the test tool in the background) and sends a nodeManagementSubscriptionRequestCall targeting the DeviceDiagnosis server feature of Client A. The DUT SHALL NOT send any subscription request targeting the DeviceDiagnosis feature of Client B within 120s.
4	The test tool responds to the subscription request with a resultData message (ACK) indicating errorNumber = 0.	The DUT remains silent.

Table 30: TC_SPINE_RTS_003 - Test steps

4.8.4 TC_SPINE_RTS_004: Ignore unknown payload Elements

Test case ID: TC_SPINE_RTS_004

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-COMP-02], [SPINE-TS-COMP-03]

Description: This test case verifies that the DUT (CS) correctly processes runtime communication even if the payload contains an entirely unknown Element. To comply with SPINE protocol rules for forward compatibility, the test tool simulates a future release by setting its SPINE specification version to a higher minor/revision value within the same major release than any currently released version. The DUT MUST silently ignore the unknown Element and process the valid Use Case data. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP Scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_AsFutureDevice
2. PRE_SPINE_ConnectionEstablished
3. PRE_SPINE_Scenario_LpcLpp_S1_S3

Step	Action/Execution	Expected result
1	The test tool sends a deviceDiagnosisHeartbeatData notify using PAR_notifyAck.	The DUT responds with a resultData message indicating errorNumber = 0 (ACK).
2	The test tool sends a loadControlLimitListData write request using PAR_writeLpcLppUnknownElement.	The DUT ignores the unknown Element and responds with a resultData message indicating errorNumber = 0 (ACK).

Table 31: TC_SPINE_RTS_004 - Test steps

4.8.5 TC_SPINE_RTS_005: Apply RFE merge logic ScaledNumberType

Test case ID: TC_SPINE_RTS_005

Role: DUT: Server (CS). Test tool: Client (EG).

Reference: [SPINE-TS-RT-03]

Description: This test case verifies that the DUT strictly applies the Restricted Function Exchange (RFE) merge logic for the ScaledNumberType when Elements are omitted. To verify this behavior in the power limitation context (LPC or LPP), the test tool (EG) sends a partial write for a LoadControl limit containing a new number but missing the scale Element. The DUT (CS) MUST accept and process this update without rejecting the request.

Applicability: Applicable to DUTs that assume the role of Controllable System (CS) in LPC or LPP Scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_SPINE_ConnectionEstablished
2. PRE_SPINE_Scenario_LpcLpp_S1_S3

Step	**Action/Execution**	**Expected result**
1	The test tool sends a deviceDiagnosisHeartbeatData notify using PAR_notifyAck.	The DUT responds with a resultData message indicating errorNumber = 0 (ACK).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Step	Action/Execution	Expected result
2	The test tool sends a loadControlLimitListData read request.	The DUT responds with a loadControlLimitListData reply. The test tool reads the value.scale of the limit and caches it internally as ${CACHE.Current_Limit_Scale}.
3	The test tool sends a loadControlLimitListData write request using PAR_writeLpcLppMissingScale.	The DUT accepts the request by sending a resultData message (ACK) indicating errorNumber = 0.
4	The test tool sends a loadControlLimitListData read request.	The DUT responds with a loadControlLimitListData reply.
5	The test tool evaluates the payload of the reply.	The test tool verifies that the newly stored limit value mathematically equals ${SYS.LpcLpp_Test_Value2_Number} * $10^{\${CACHE.Current_Limit_Scale}}$.

Table 32: TC_SPINE_RTS_005 - Test steps

4.9 Runtime behavior of client

4.9.1 TC_SPINE_RTC_001: Ignore extra server Elements

Test case ID: TC_SPINE_RTC_001

Role: DUT: Client (EG). Test tool: Server (CS).

Reference: [SPINE-TS-COMP-03]

Description: This test case verifies that the DUT correctly processes incoming communication even if the server's reply payload contains an additional, standard SPINE Element (e.g., "label") that is not explicitly required by the Use Case. The DUT MUST silently ignore the irrelevant Element and proceed with the control flow. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The DUT assumes the role of an Energy Guard (EG) and the test tool simulates a Controllable System (CS).

Applicability: Applicable to DUTs that assume the role of Energy Guard (EG) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_ReplyLpcLppExtraElement_Configured
2. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool awaits/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s.

Table 33: TC_SPINE_RTC_001 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.9.2 TC_SPINE_RTC_002: Ignore unknown server Elements

Test case ID: TC_SPINE_RTC_002

Role: DUT: Client (EG). Test tool: Server (CS).

Reference: [SPINE-TS-COMP-02], [SPINE-TS-COMP-03]

Description: This test case verifies that the DUT (Energy Guard) correctly processes incoming communication even if the server's reply payload contains an entirely unknown Element. To comply with SPINE protocol rules for forward compatibility, the test tool simulates a future release by setting its SPINE specification version to a higher minor/revision value within the same major release than any currently released version. The DUT MUST silently ignore the unknown Element and proceed with the control flow by successfully completing the initial communication. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The DUT assumes the role of an Energy Guard (EG) and the test tool simulates a Controllable System (CS).

Applicability: Applicable to DUTs that assume the role of Energy Guard (EG) in LPC or LPP scenarios.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_ReplyLpcLppUnknownElement_Configured
2. PRE_TestTool_AsFutureDevice
3. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool awaits/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s.

Table 34: TC_SPINE_RTC_002 - Test steps

4.9.3 TC_SPINE_RTC_003: Ignore useCaseAvailable flag

Test case ID: TC_SPINE_RTC_003

Role: DUT: Client (EG). Test tool: Server (CS).

Reference: [SPINE-TS-UCD-01]

Description: This test case verifies that the DUT completely ignores the useCaseAvailable flag if it is sent by the remote communication partner during Use Case discovery. Even if the flag is explicitly set to false, the DUT MUST silently ignore it and proceed with the control flow by successfully initiating the required node management operations. To verify this mechanism, the test utilizes a power limitation context (LPC or LPP). The DUT assumes the role of an Energy Guard (EG) and the test tool simulates a Controllable System (CS).

Applicability: Applicable to DUTs that assume the role of Energy Guard (EG) in LPC or LPP scenarios.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test execution: This test case SHALL be executed exactly once, utilizing one of the power limitation Use Cases (LPC or LPP) supported by the DUT. The test tool SHALL dynamically select the Use Case and resolve the corresponding parameters and preconditions accordingly.

Preconditions:

1. PRE_TestTool_ReplyUcdAvailableFalse_Configured
2. PRE_SPINE_NewConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool awaits/supports PRE_SPINE_Scenario_LpcLpp_S1_S3.	The DUT successfully completes PRE_SPINE_Scenario_LpcLpp_S1_S3 within 120s.

Table 35: TC_SPINE_RTC_003 - Test steps

Copyright © 2026 EEBus Initiative e.V. All rights reserved.